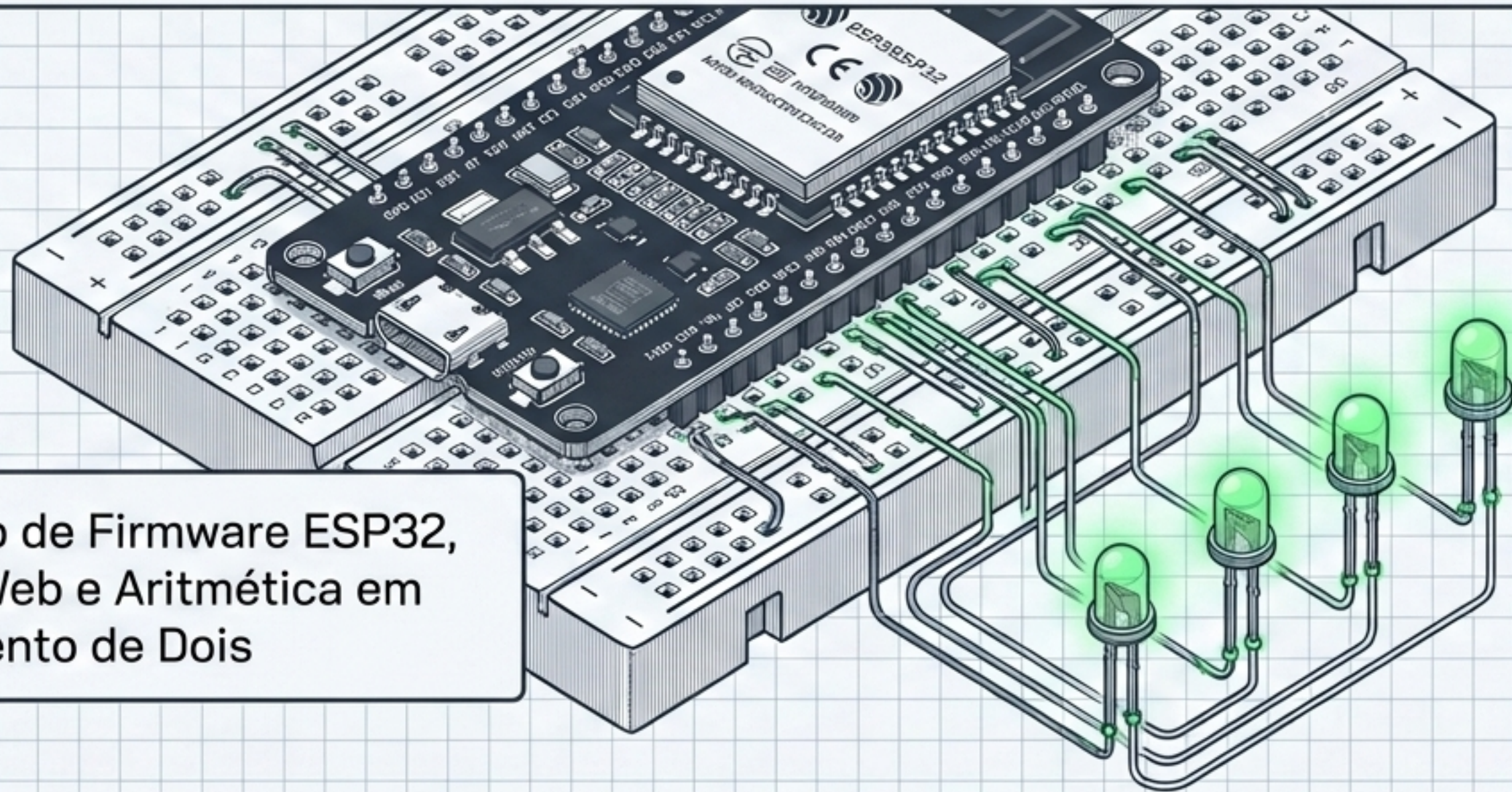


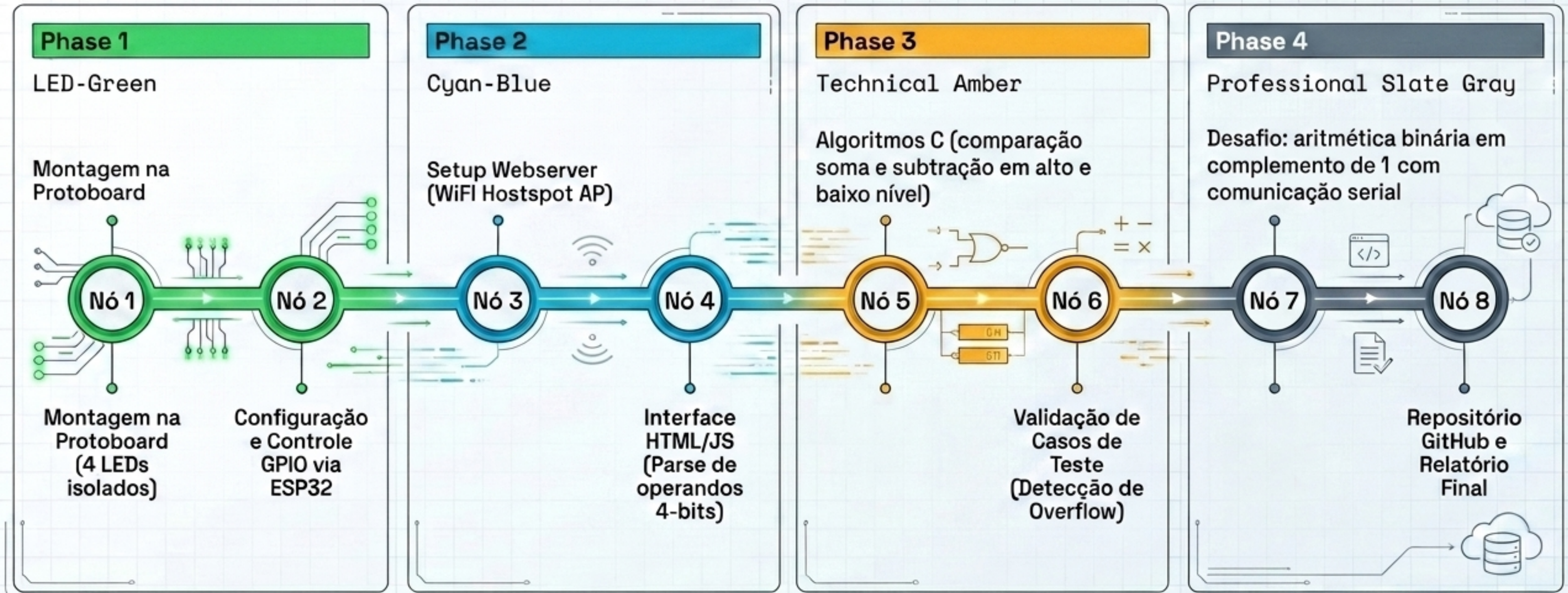
Calculadora de 4 Bits

Planejamento de Laboratório: PCS3732

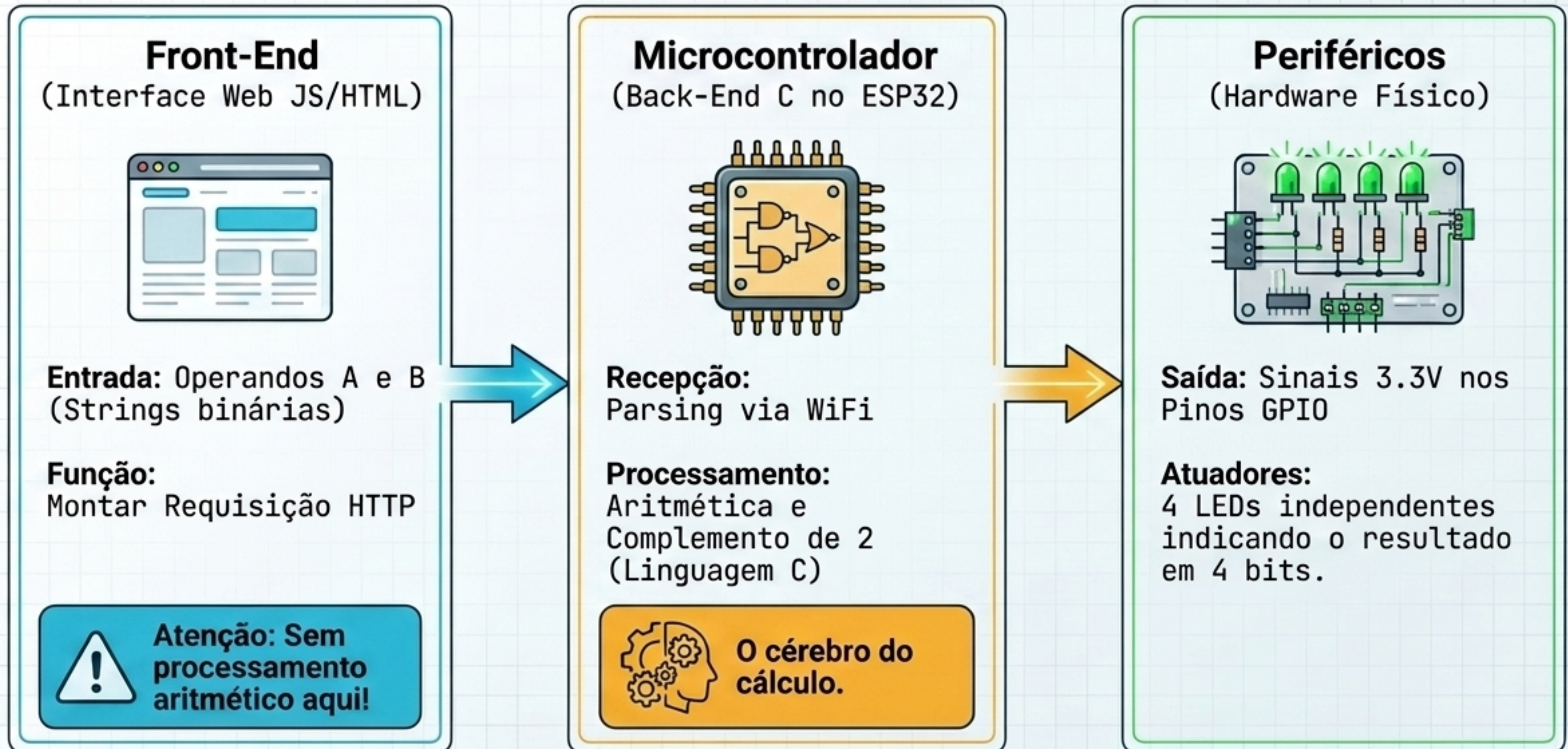


Integração de Firmware ESP32,
Servidor Web e Aritmética em
Complemento de Dois

0 Pipeline de Implementação



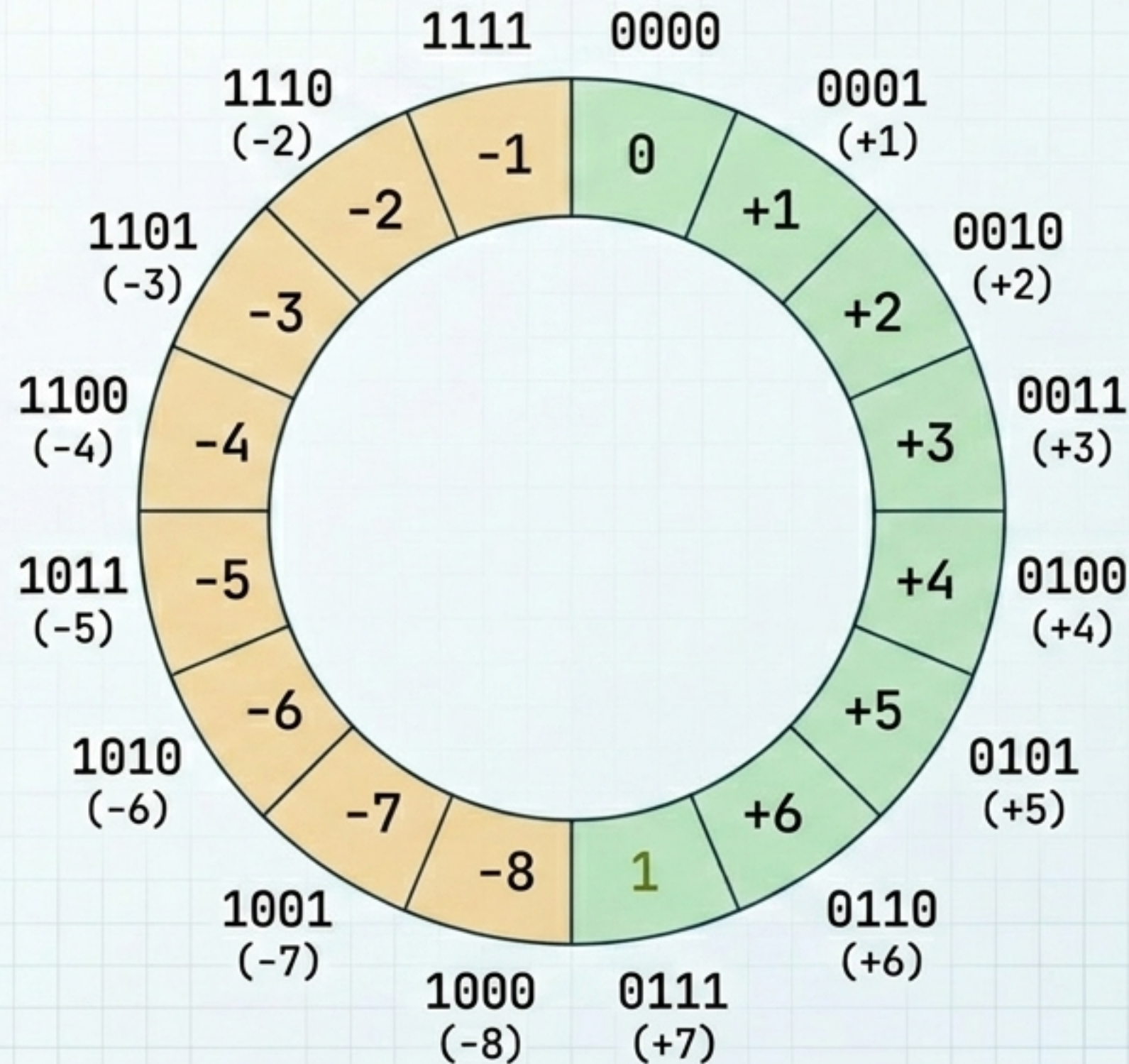
Arquitetura do Sistema: Divisão de Responsabilidades



A Matemática: O Espectro do Complemento de Dois

A representação de 4 bits permite 16 estados únicos.

Bit de Sinal: O Bit Mais Significativo (MSB) dita a polaridade (0 = Positivo, 1 = Negativo).



Como inverter o sinal?

Passo 1: Inverter todos os bits (Operador ~).

Passo 2: Somar 1 binário.

Exemplo:

0011 (+3) -> 1100
-> 1101 (-3).

Matriz de Tradução: C vs. RISC-V Assembly

Nível de Firmware (C no ESP32)

1 Declaração estrita de tipos para forçar limites de
2 memória:

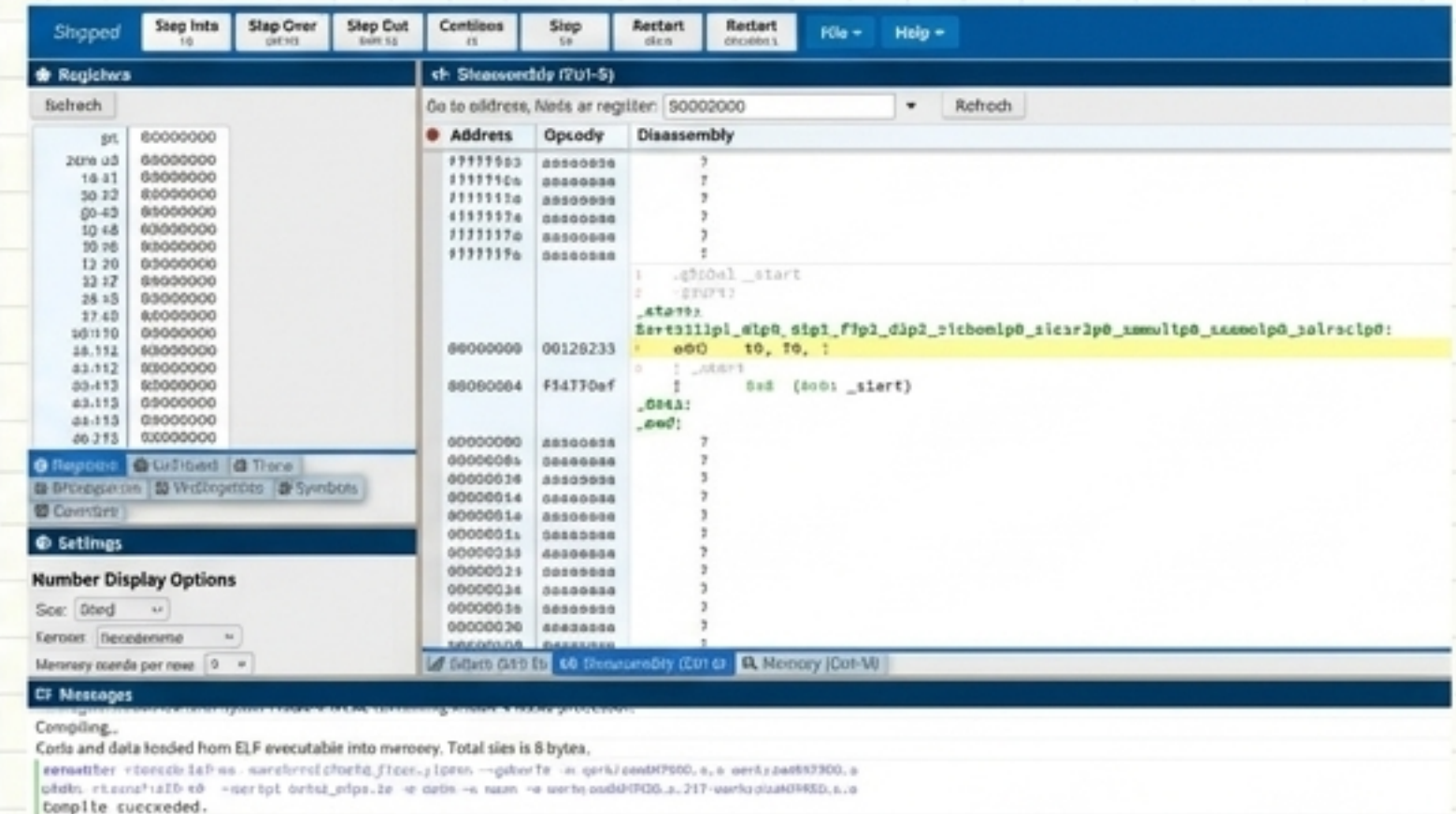
3
4 `int8_t a, b, res;`

5
6 Subtração abstrata:

7
8 `res = (a - b) & 0x0F;`

9
10 O compilador gerencia o bit de sinal e o
complemento.

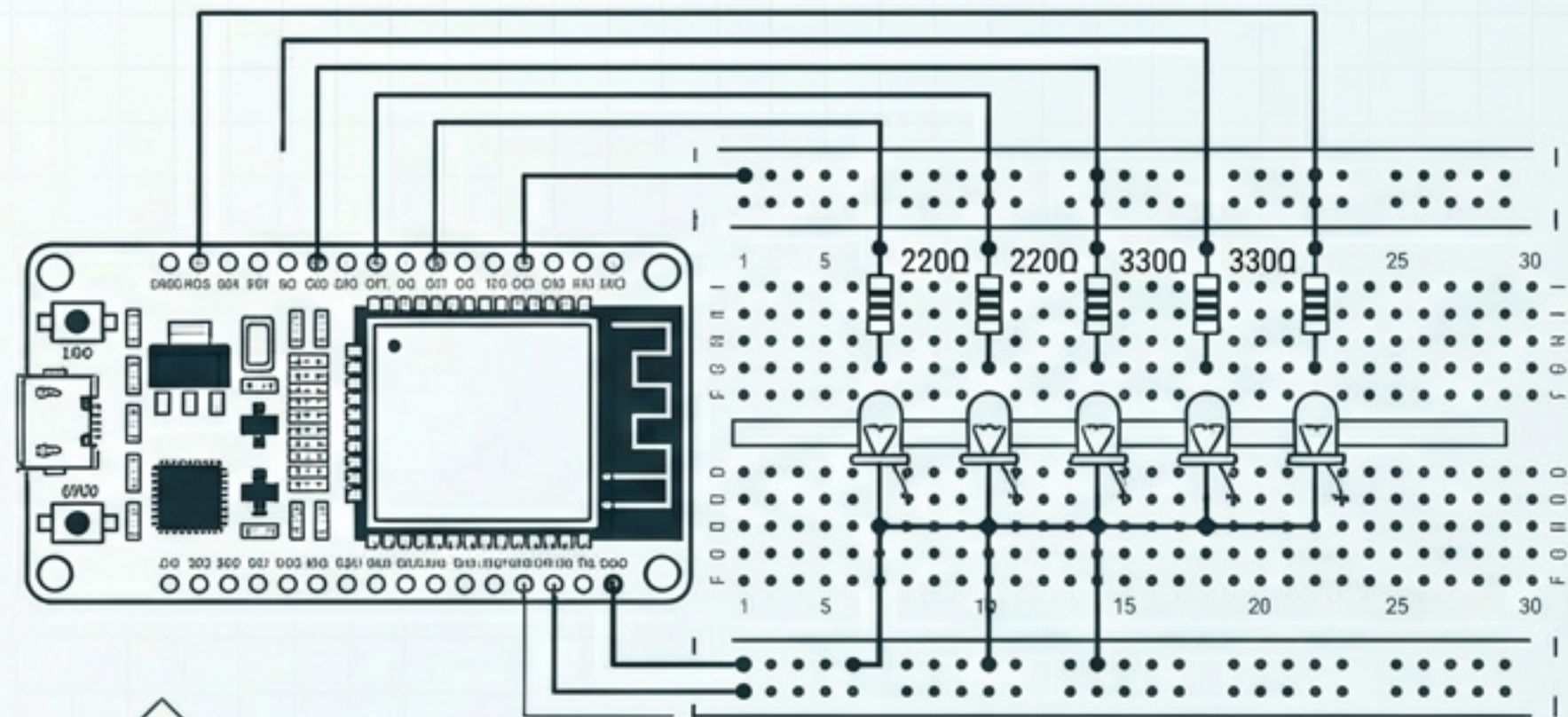
Nível de Arquitetura (RISC-V)



- Acesso direto aos registradores base: x10, x11, x12
- Instrução nativa: `sub x12, x10, x11`
- A ULA (Unidade Lógica Aritmética) processa automaticamente o complemento em hardware, tratando todos os dados como complemento de dois.

Etapa 1: Validação Física na Protoboard

Regra de Ouro: Isole os domínios de falha.



O Teste Cego

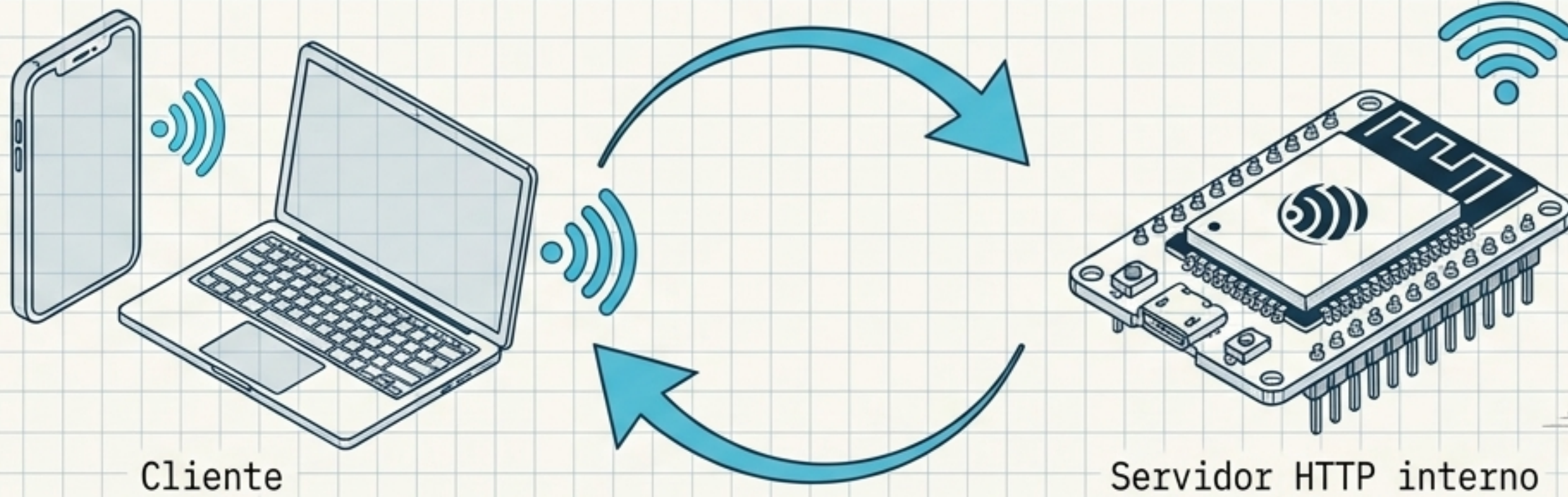
Ligue os **LEDs** e **resistores** de limitação de corrente diretamente no **3.3V** e **GND** para garantir que nenhum diodo está queimado.

Mapeamento GPIO

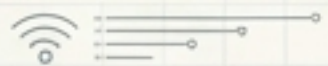
Conecte os LEDs aos pinos **GPIO** do ESP32.

Escreva um script básico de **Blink sequencial** (Bit shift) apenas para atestar atestar a **condutividade** e a **integridade** da solda/jumpers.

0 ESP32 como Webserver Autônomo



Step 1: Access Point Mode (WIFI_AP)



O ESP32 cria sua própria rede (SSID: `Calculadora_ESP32`).

Nenhuma conexão externa com a Internet é necessária.

Step 2: HTTP Server Setup



O microcontrolador abre a porta 80 e aguarda conexões.

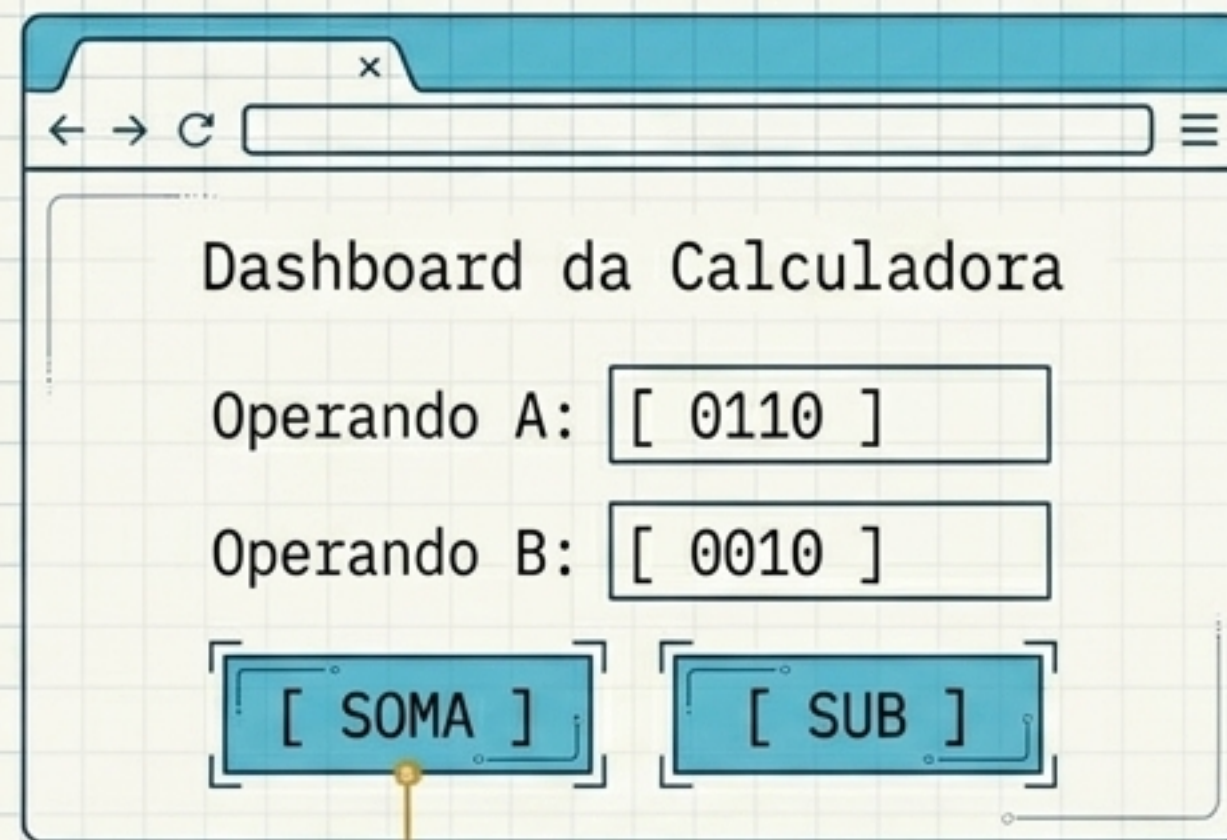
Step 3: Resposta Síncrona

(Referência: [RandomNerdTutorials](#)).

O ESP32 despacha a interface HTML embutida em sua memória Flash diretamente para o navegador do cliente.



Front-End: Interface e Construção de Payload



Dashboard da Calculadora

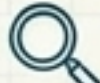
Operando A: [0110]

Operando B: [0010]

[SOMA] [SUB]

Mecânica JS

O script local captura os formulários e realiza um GET Request dinâmico:

 `http://192.168.4.1/calc?a=0110&b=0010&op=add`

A formatação e o gatilho da requisição ocorrem aqui. Todo o trabalho analítico é remetido ao servidor embarcado.

Firmware C: Do String Web ao Pulso Elétrico

// 1. Parsing: String para Inteiro

```
int valA = strtol(paramA.c_str(), NULL, 2);
```

// 2. Operação Aritmética (C nativo)

```
int resultado = (op == "add") ? (valA + valB) : (valA - valB);
```

// 3. Mascaramento (Garantindo 4 bits)

```
resultado = resultado & 0x0F;
```

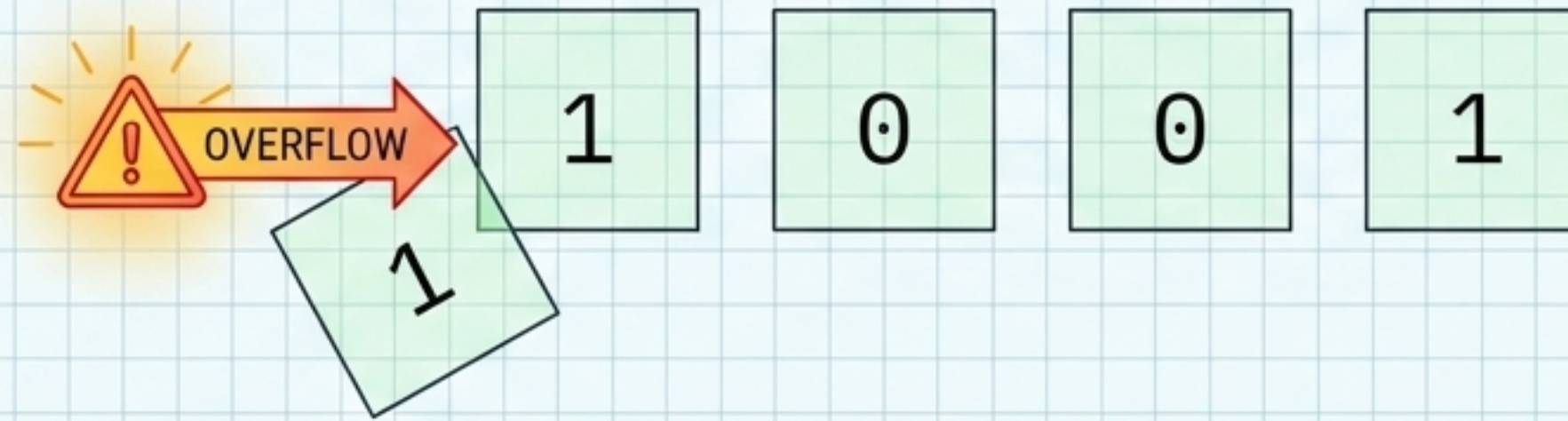
// 4. Output para GPIO

```
digitalWrite(LED_BIT0, resultado & 0x01);
```

Evidência de Relatório:

Documento detalhadamente estes trechos. Eles atestam que a computação ocorreu nativamente na arquitetura do ESP32.

Anatomia do Overflow (Estouro de Capacidade)

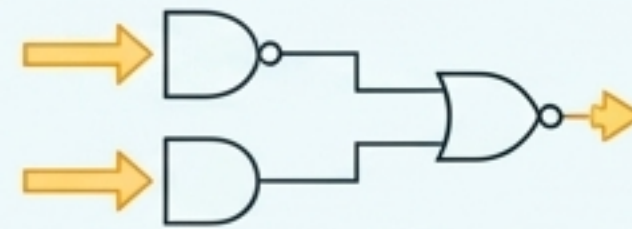


O Problema

$0101 (+5) + 0100 (+4)$
 $= 1001 (-7 \text{ no comp. de } 2)$

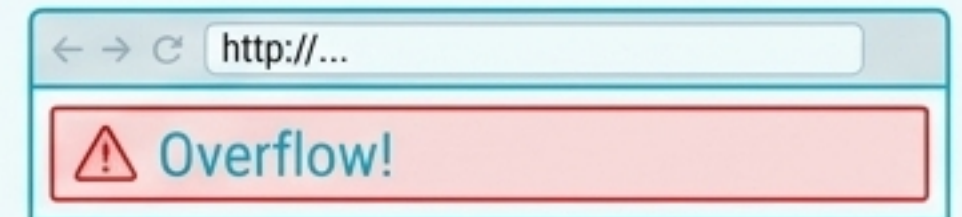
O hardware realizou a soma binária perfeita, mas a interpretação humana como complemento de dois falhou. Ocorreu invasão do bit de sinal.

A Solução: Detecção no ESP32



Se a soma de dois positivos gera um negativo (ou vice-versa), ocorreu Overflow.

O Sinal



O ESP32 deve retornar um código de erro no cabeçalho HTTP ou injetar um `<div class="alert">Overflow!</div>` na atualização da página.

Tabela de Validação Experimental

Planeje os testes para cobrir os limites do complemento de dois (-8 a +7).

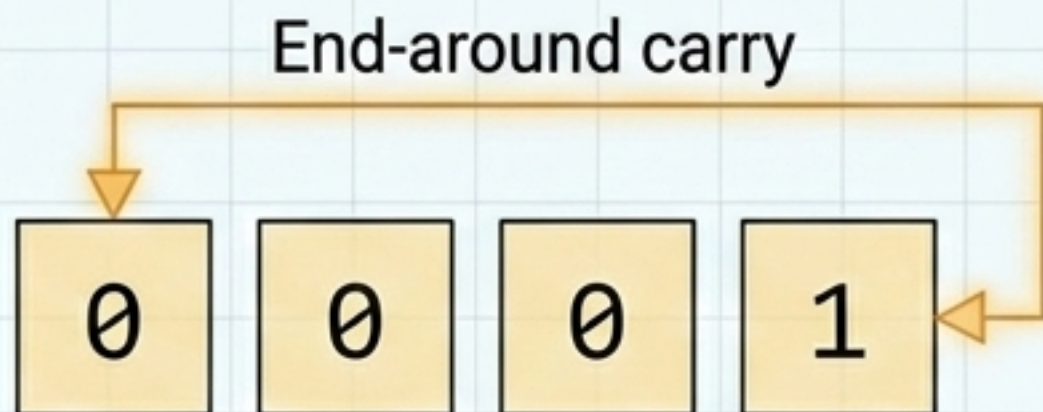
Op	Entrada A	Entrada B	LEDs Esperados	Status/Overflow
SOMA	0011 (+3)	0010 (+2)	0101 (ON, OFF, ON, OFF)	OK
SUB	0111 (+7)	0001 (+1)	0110 (ON, ON, OFF, OFF)	OK
SOMA	0111 (+7)	0010 (+2)	1001 (Led sinal ON)	OVERFLOW! ⚠
SUB	1000 (-8)	0001 (+1)	0111 (Led sinal OFF)	OVERFLOW! ⚠



Nota: Fotografe e colete screenshots de cada estado no laboratório para as evidências do relatório.

O Desafio: Complemento de Um & I/O Serial

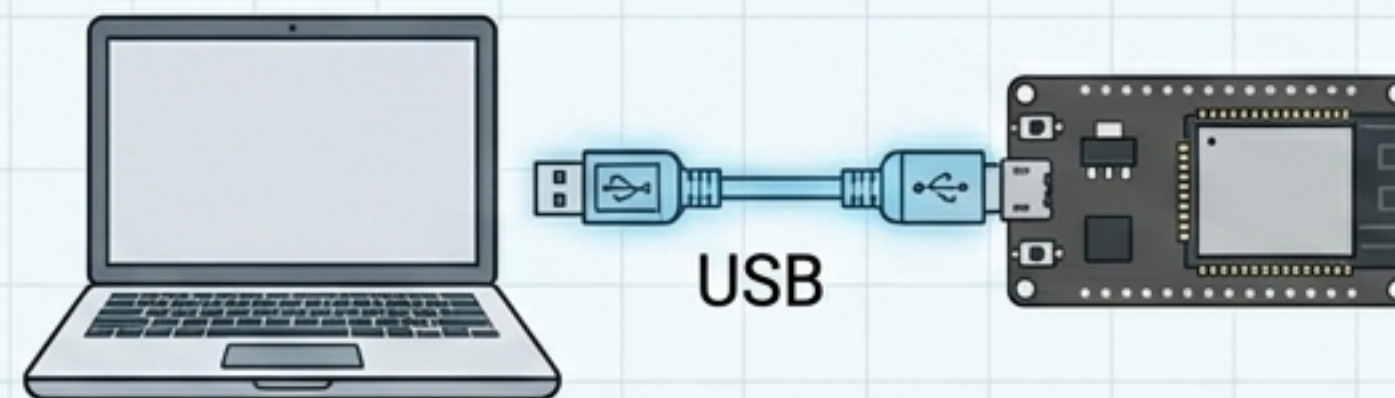
Mudança Matemática



No Complemento de Um, o zero tem duas representações (0000 e 1111).

End-around carry: Se houver vai-um (carry out) do bit mais significativo na soma, este carry deve ser somado de volta ao bit menos significativo do resultado.

Mudança de Interface



Remova completamente o servidor Web. O ESP32 não transmite via WiFi.

Utilize o **Serial Monitor** (ex: 115200 baud).

Entrada via teclado no computador conectando à USB do ESP32, lendo através da função `Serial.readString()`.

Método de Trabalho: Recomendações da Disciplina



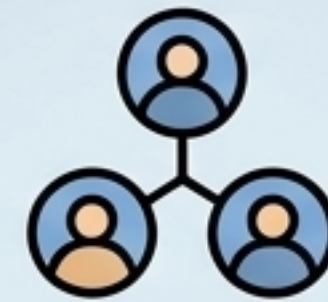
Controle de Versão (Git)



Depuração Incremental



Isolamento de Erros



Método do Grupo

Divida as tarefas, mas faça integração de código com avaliação por pares no laboratório.

Checklist de Entrega do Relatório Final

Certifique-se de que o documento consolida o Planejamento e os Resultados experimentais.

- [] **Formatação Estrita:** Fonte **Arial 12**.
- [] **Extensão:** Máximo absoluto de **10 páginas**.
- [] **Formato de Arquivo:** Submissão exclusiva em **.PDF**.
- [] **Repositório de Código:** Link explícito para o **GitHub** contendo o código **C** do firmware e HTML/JS embarcado. O repositório **DEVE** ser **Público!**
- [] **Evidências Visuais:** Inclusão de fotos da protoboard funcionando, capturas da interface Web, e os trechos de código **C** documentados.
- [] **Template:** Lembrar de utilizar o template fornecido no moodle!

